

# Sécurisation d'Actions Destructrices avec -WhatIf et ShouldProcess

## 1. Cadrage Opérationnel

L'objectif de cet atelier est de concevoir un outil d'administration sécurisé nommé **Arreter-ProcessusNonConforme.ps1**.

Ce script doit être capable d'interrompre des processus suspects ou trop gourmands en mémoire vive, tout en intégrant nativement la protection du moteur PowerShell :

- Simulation sans risque via le commutateur universel **-WhatIf**
- Demande d'accord interactif via **-Confirm**
- Évaluation unitaire au fil de l'eau pour les objets transmis via le pipeline (|)
- Journalisation étanche (aucun faux positif en mode simulation)

## 2. Instructions de Réalisation du Script (Arreter-ProcessusNonConforme.ps1)

Développez votre script en suivant rigoureusement les étapes ci-dessous.

### Étape 1 : En-tête et Déclaration des Paramètres

#### 1. Activation de la liaison avancée :

- Déclarez l'attribut de liaison de commande avancée en activant la prise en charge de l'interception (SupportsShouldProcess = \$true).
- Calibrez le niveau de criticité par défaut sur un impact moyen (ConfirmImpact = 'Medium').

#### 2. Documentation intégrée (*Comment-Based Help*) :

- Rédigez un bloc de commentaires structuré (<# ... #>) au-dessus des paramètres comprenant :
  - .SYNOPSIS : résumé de l'outil en une phrase.
  - .DESCRIPTION : détail du fonctionnement (seuil mémoire, filtrage, journalisation).
  - .PARAMETER NomsProcessus : description de la liste des exécutable ciblés.
  - .PARAMETER SeuilMemoireMo : description du filtre volumétrique en mégaoctets.
  - .PARAMETER CheminLog : emplacement du journal d'audit.
  - .EXAMPLE : au moins deux exemples réalistes d'appel (en mode nominal et avec -WhatIf).

#### 3. Bloc de paramètres (param) :

- Définissez trois paramètres typés :
  - [string[]]\$NomsProcessus : tableau de noms de processus (valeur par défaut : au moins deux exécutable de test, par exemple "notepad", "calc"). Ce paramètre doit

être obligatoire et accepter les données depuis le pipeline par valeur (ValueFromPipeline = \$true).

- [int]\$SeuilMemoireMo = 50 : seuil minimum de mémoire de travail en mégaoctets en dessous duquel un processus est ignoré.
- [string]\$CheminLog = "C:\Audit\processus\_arretes.log" : chemin complet du fichier journal.

## Étape 2 : Bloc d'Initialisation (begin)

### 1. Métrologie :

- Initialisez trois compteurs numériques à zéro :
  - Compteur des instances analysées
  - Compteur des instances effectivement arrêtées
  - Compteur des instances préservées (soit sous le seuil mémoire, soit refusées/simulées)

### 2. Préparation de l'environnement :

- Vérifiez si le répertoire de destination du journal d'audit (ex: C:\Audit) existe sur le disque. S'il est absent, créez-le automatiquement.

### 3. Traçabilité :

- Émettez un message informatif via le canal verbeux (Write-Verbose) signalant le démarrage de l'audit et rappelant le seuil mémoire configuré.

## Étape 3 : Traitement Unitaire et Interception (process)

Le bloc process traite unitairement chaque nom de processus transmis directement ou via le pipeline.

Pour chaque élément reçu :

### 1. Recherche des instances actives :

- Récupérez les instances en cours d'exécution associées à ce nom.
- Masquez les erreurs non-bloquantes si le programme n'est pas lancé sur la machine.

### 2. Évaluation de chaque instance trouvée :

- Incrémentez le compteur des instances analysées.
- Calculez la mémoire vive consommée (WorkingSet64) convertie en mégaoctets et arrondie à une décimale.
- **Contrôle du seuil :**
  - Si la mémoire est inférieure au seuil \$SeuilMemoireMo : ignorez le processus, incrémentez le compteur des préservés et tracez l'information en verbeux.
  - Si la mémoire franchit le seuil : préparez l'interception d'action.

### 3. Interception par le moteur (\$PSCmdlet.ShouldProcess) :

- Construisez deux chaînes de texte explicites :
  - **Cible** : Indiquant le nom de l'exécutable, son PID et sa mémoire en Mo.
  - **Action** : Décrivant l'arrêt immédiat et la purge de l'espace mémoire.
- Soumettez l'opération à \$PSCmdlet.ShouldProcess(\$Cible, \$Action).

4. **Branche Confirmée (if positif — action réelle validée) :**
  - Arrêtez physiquement le processus cible.
  - **Consigne anti-double prompt :** Neutralisez formellement toute demande de confirmation supplémentaire de la part de la commande d'arrêt interne.
  - Incrémentez le compteur d'arrêts effectifs.
  - Inscrivez une ligne horodatée au format [yyyy-MM-dd HH:mm:ss] [ARRET] PID: ... | Nom: ... | RAM: ... dans le fichier journal avec un encodage strict UTF-8.
  - Affichez une confirmation visuelle dans la console (ex: en vert).
5. **Branche Alternative (else — simulation -WhatIf ou refus opérateur -Confirm) :**
  - Incrémentez le compteur d'instances préservées.
  - Émettez une notification via le canal verbeux indiquant que le processus a été laissé actif.
  - **Règle d'or :** N'écrivez **aucune ligne** dans le fichier journal et n'affichez aucun message de succès dans la console.

## Étape 4 : Synthèse Structurée (end)

Le script ne doit pas se terminer par de simples chaînes de texte volatiles.

1. **Détection du mode d'exécution :**
  - Évaluez la variable de portée locale \$WhatIfPreference pour déterminer si le script a tourné en mode simulation.
2. **Émission de l'objet de bilan :**
  - Créez et émettez dans le pipeline de sortie standard un objet personnalisé [PSCustomObject] contenant les propriétés suivantes :
    - DateExecution : date et heure de fin d'audit.
    - InstancesVerifiees : nombre total d'instances examinées.
    - InstancesArretees : nombre de processus réellement neutralisés.
    - InstancesPreservees : nombre de processus épargnés (ou simulés).
    - SeuilAppliqueMo : valeur du seuil mémoire utilisé.
    - ModeSimulation : valeur booléenne indiquant si -WhatIf était activé.

## 3. Protocole de Test

Pour valider le comportement de votre script, démarrez 2 ou 3 instances de blocs-notes (notepad.exe) en arrière-plan.

### Test 1 : Validation du Pipeline et de la Simulation (-WhatIf)

Exécutez la commande suivante :

```
"notepad" | .\Arreter-ProcessusNonConforme.ps1 -SeuilMemoireMo 0 -WhatIf
```

- **Résultats obligatoires :**
  - Des lignes What if: Performing the operation... s'affichent pour chaque instance.

- Les processus notepad sont toujours vivants dans le gestionnaire des tâches.
- Le fichier C:\Audit\processus\_arretes.log reste strictement vide (ou n'est pas modifié).
- L'objet final affiche InstancesArretees = 0 et ModeSimulation = True.

## Test 2 : Validation du Dialogue Interactif et Anti-Double Prompt (-Confirm)

Exécutez la commande suivante :

```
.\Arreter-ProcessusNonConforme.ps1 -NomsProcessus "notepad" -SeuilMemoireMo 0 -Confirm
```

- **Résultats obligatoires :**

- Une invite de validation s'affiche pour la première instance. Répondez [N] Non. Le processus doit rester vivant et aucune ligne ne doit être écrite dans le log.
- Pour l'instance suivante, répondez [Y] Oui.
- **Contrôle clé :** Une seule et unique invite doit s'afficher par instance (aucun second prompt ne doit apparaître en sous-main).
- Le processus validé est arrêté et la ligne correspondante est consignée dans le fichier journal.

## Test 3 : Exécution Nominale Directe

Exécutez la commande suivante :

```
.\Arreter-ProcessusNonConforme.ps1 -NomsProcessus "notepad" -SeuilMemoireMo 0
```

- **Résultats :**

- Toutes les instances restantes sont arrêtées immédiatement et sans invite.
- Les lignes d'arrêt horodatées apparaissent dans C:\Audit\processus\_arretes.log.
- L'objet final retourne ModeSimulation = False et le compte exact d'instances arrêtées.